Lee Ee Xuan (Maxkedd) - Project Portfolio Page

Overview

EduReserve is a Java command-line application that helps school administrators and educators manage exam and lecture venue bookings. My contributions cover booking creation and validation, undo/redo, the initial application structure, automated testing, defensive coding and documentation.

Summary of contributions

Code contributed: RepoSense dashboard.

New feature: Booking creation (`book`)

- **Purpose:** Creates a named exam or lecture booking with its venue and timing, allowing users to record an event in one command.
- **Implementation:** Built the initial `Task`, `Booking` and `BookingList` classes and their add, retrieve, remove and count operations. Added booking names, case-insensitive command recognition, required prefix order and nonblank-field validation. Extracted booking execution into `BookCommand`, reusing `Parser` and delegating from `Ui`.
- **Correctness:** Added case-insensitive duplicate-name rejection and moved the existing clash check before insertion. Removed duplicate parsing, insertion and confirmation from `Ui`, so rejected bookings leave the list unchanged. The clash algorithm was a teammate's work; my contribution was integration and correction.
- **Evidence:** Initial structure, booking names, command extraction, correctness fixes.

New features: Undo (`/undo`) and redo (`/redo`)

- **Purpose:** Lets users recover from accidental bookings or cancellations and reapply an accidentally undone command without retyping it.
- **Implementation:** Implemented session history using before/after booking snapshots and original command text. `Ui` retains an `UndoCommand`; `RedoCommand` shares that history. Undo restores the previous list and original booking order, while redo restores the resulting list and returns the entry to undo history.
- **History rules:** Only successful `book` and `cancel` operations are recorded. A successful new booking or cancellation clears redo history; failed and read-only commands preserve it. Added repeated undo/redo, empty-history feedback and argument validation. The existing cancellation feature was a teammate's work; I made it undoable.
- **Integration and limits:** Restored the retained history instance after a merge removed it. History is session-only, and restoring a whole-list snapshot can overwrite intervening edits; I documented these limits for users.
- **Evidence:** Undo, redo, integration repair.

Overall code structure, separation of class roles and clarity

- **Foundation:** Established the initial application structure: `EduReserve` runs the console loop, `Ui` coordinates commands and feedback, `Parser` interprets and validates input, `Task`/`Booking` represent booking data, and `BookingList` manages the collection. This gave teammates clear locations for extending the application.
- **Separation of responsibilities:** Moved booking execution into `BookCommand` to reduce duplicated logic in `Ui`. Kept parsing separate from booking-list mutation and reused the shared booking state and parser rather than maintaining competing copies.
- **Readability:** Added explanatory Javadoc and assertions, and documented class responsibilities and dependencies through the overall class diagram. These changes make the execution flow and extension points easier for future developers to understand. This describes my initial structure and later refinements, not authorship of every class in the final application.
- **Evidence:** Application foundation, BookCommand extraction, class diagram.

Testing and defensive coding

- **Automated regression tests:** Added and expanded JUnit tests for booking commands, the booking model, parsing, UI behaviour and the application loop. Extended tests for teammates' list, find, calendar and storage functionality to check interactions and failure paths, without claiming those features as my implementations.
- **State preservation:** Tested that malformed input, duplicate names and booking clashes do not insert a booking or alter existing data. Added undo/redo tests for repeated operations, restored order, cancellation, empty history, redo invalidation after a successful new command, and history preservation after failed or read-only commands.
- **Defensive checks:** Added assertions for booking fields, parser input, non-null dependencies and `BookingList.add` invariants, including size changes and append position. Enabled assertions for Gradle `run`. Assertions detect programming errors; invalid user commands are handled through exceptions and informative feedback so the application can continue.
- **Diagnostics:** Added diagnostic logging for booking rejection/success and undo/redo history changes, with tests for those logging paths. Expanded parser, redo and storage failure-path tests and added Gradle coverage reporting/verification configuration.
- **Evidence:** Assertions, regression tests, undo tests, redo tests, logging and coverage.

Documentation: User Guide contributions

- Wrote the **Creating a booking**, **Undoing a command** and **Redoing a command** sections, including command formats, examples, expected results and corresponding command-summary entries.
- Explained required booking fields, unique names, time-slot clashes and optional attendee counts. Documented undo/redo eligibility, empty-history behaviour, session-only history, redo invalidation and the effect of snapshot restoration on intervening edits.
- **Evidence:** User Guide, authored sections.

Documentation: Developer Guide contributions

- Documented the **book**, **undo** and **redo** implementation flows and added their sequence diagrams. Explained validation, shared history, before/after snapshots and restoration behaviour to help developers trace execution.
- Added the **overall class diagram** and its explanation of package grouping, class roles, inheritance and selected dependencies. Refined abstract-class/method labels and later added the command class diagram and clarified UML notation.
- Revised sequence diagrams across the guide for consistent participants, calls, returns and output notation. These were consistency improvements to shared documentation; teammates remain the authors of their own feature designs and original diagrams.
- Contributed user stories and product-scope text, providing context for the implementation.
- **Evidence:** Developer Guide, book diagram, user stories, diagram consistency and scope, UML refinements.